

Toate cele trei metode rezolva sistemul $Ax = b$ bazandu-se pe descompunerea matricei sub forma $A = M - N$.

Relatia generala de iteratie este: $x^{(k+1)} = M^{-1}Nx^{(k)} + M^{-1}b$

Pentru a le diferentia, spargem matricea sistemului in trei componente: $A = D - L - U$ (unde D este diagonala, -L este triunghiul de sub diagonala, iar -U este triunghiul de deasupra diagonalei).

```
n = 500000;
tol = 1e-6;
nitmax = 500;
x0 = zeros(n, 1);

e = ones(n, 1);
A = spdiags([-e, -e, 5*e, -e, -e], [-3, -1, 0, 1, 3], n, n);
A(1, n) = 1;
A(n, 1) = 1;

b = ones(n, 1);
b(1) = 4; b(n) = 4;
b(2) = 2; b(n-1) = 2;
b(3) = 2; b(n-2) = 2;

sol_exacta = ones(n, 1);

[x_jac, iter_jac] = Jacobi(A, b, x0, tol, nitmax);

eroare_jac = norm(x_jac - sol_exacta, inf);
fprintf('Jacobi: Iteratii: %d | Eroare fata de solutia exacta: %e\n\n', iter_jac,
eroare_jac);
```

Jacobi: Iteratii: 56 | Eroare fata de solutia exacta: 3.741444e-06

```
[x_gs, iter_gs] = GaussSeidel(A, b, x0, tol, nitmax);
eroare_gs = norm(x_gs - sol_exacta, inf);
fprintf('Gauss-Seidel: Iteratii: %d | Eroare fata de solutia exacta: %e\n\n',
iter_gs, eroare_gs);
```

Gauss-Seidel: Iteratii: 33 | Eroare fata de solutia exacta: 1.545213e-06

```

omega = 1.15;
[x_sor, iter_sor] = SOR(A, b, omega, x0, tol, nitmax);

eroare_sor = norm(x_sor - sol_exacta, inf);
fprintf('SOR: Iteratii: %d | Eroare fata de solutia exacta: %e\n\n', iter_sor,
eroare_sor);

```

SOR: Iteratii: 25 | Eroare fata de solutia exacta: 9.423199e-07

Metoda izoleaza strict diagonala principala. Componentele noului vector se calculeaza complet independent, bazandu-se exclusiv pe valorile de la pasul complet anterior. Este simpla si usor de paralelizat, dar adesea are cea mai lenta convergenta.

Descompunere: $M = D$ si $N = L + U$

Formula: $x^{(k+1)} = D^{-1}(L + U)x^{(k)} + D^{-1}b$

```

function [x, ni] = Jacobi(A, b, x0, err, nitmax)
if nargin < 5, nitmax = 500; end
if nargin < 4, err = 1e-6; end
if nargin < 3, x0 = zeros(size(b)); end

[m, n] = size(A);
if m ~= n || length(b) ~= m
    error('A must be square and length(b) must equal size(A,1).')
end

ni = 0;
M = spdiags(diag(A), 0, n, n);
N = M - A;
x = x0(:);

for i = 1:nitmax
    x0 = x;
    x = M \ (N * x0 + b);

    dx = norm(x - x0, inf);
    xn = norm(x, inf);
    if xn > 0
        cond = dx < err * xn;
    else
        cond = dx < err;
    end

    if cond
        ni = i;
        return
    end
end

```

```
end
```

```
ni = nitmax;  
warning('Jacobi:MaxIter', 'Maximum number of iterations (%d) reached without  
convergence.', nitmax)  
end
```

Matricea M include acum si partea inferior triunghiulara. Algoritmic, asta inseamna ca metoda foloseste valorile noi pentru elementele solutiei imediat ce au fost calculate la pasul curent $k + 1$. Aceasta propagare rapida a informatiei asigura o convergenta mai buna decat la Jacobi.

Descompunere: $M = D - L$ si $N = U$

Formula: $x^{(k+1)} = (D - L)^{-1}Ux^{(k)} + (D - L)^{-1}b$

```
function [x, ni] = GaussSeidel(A, b, x0, err, nitmax)  
if nargin < 5, nitmax = 500; end  
if nargin < 4, err = 1e-6; end  
if nargin < 3, x0 = zeros(size(b)); end  
  
[m, n] = size(A);  
if m ~= n || length(b) ~= m  
    error('A must be square and length(b) must equal size(A,1).')  
end  
  
M = tril(A);  
  
N = M - A;  
  
x = x0(:);  
ni = 0;  
  
for k = 1:nitmax  
    x_old = x;  
  
    x = M \ (N * x_old + b);  
  
    dx = norm(x - x_old, inf);  
    xn = norm(x, inf);  
    if xn > 0  
        cond = dx < err * xn;  
    else  
        cond = dx < err;  
    end  
  
    if cond  
        ni = k;  
        return  
    end  
end
```

```

    end
end

ni = nitmax;
warning('GaussSeidel:MaxIter', 'Maximum number of iterations (%d) reached without
convergence.', nitmax)
end

```

SOR este o extensie a metodei Gauss-Seidel, introducand un parametru de relaxare $\omega \in (1, 2)$. Algoritmul face o combinatie liniara intre solutia veche si pasul Gauss-Seidel pentru a forta supra-relaxarea, ceea ce scade drastic numarul de iteratii necesare pana la solutia finala.

Descompunere: $M = \frac{1}{\omega}D - L$ si $N = \frac{1-\omega}{\omega}D + U$

Formula: $x^{(k+1)} = (1 - \omega)x^{(k)} + \omega \cdot x_{GaussSeidel}^{(k+1)}$

```

function [x, ni] = SOR(A, b, omega, x0, err, nitmax)
if nargin < 6, nitmax = 500; end
if nargin < 5, err = 1e-6; end
if nargin < 4 || isempty(x0), x0 = zeros(size(b)); end
if nargin < 3 || isempty(omega), omega = 1.15; end

[m, n] = size(A);
if m ~= n || length(b) ~= m
    error('A must be square and length(b) must equal size(A,1).')
end

D = spdiags(diag(A), 0, n, n);
L = -tril(A, -1);
U = -triu(A, 1);

M = (1/omega) * D - L;
N = ((1-omega)/omega) * D + U;

x = x0(:);
ni = 0;

for k = 1:nitmax
    x_old = x;

    x = M \ (N * x_old + b);

    dx = norm(x - x_old, inf);
    xn = norm(x, inf);
    if xn > 0
        cond = dx < err * xn;
    end
end

```

```
else
    cond = dx < err;
end

if cond
    ni = k;
    return
end
end

ni = nitmax;
warning('SOR:MaxIter', 'Maximum number of iterations (%d) reached without
convergence.', nitmax)
end
```